

Multi-Tier Computing: Architecture, Thin Clients & Design

Unit 4 – PPT 2 Exam Notes

Contents

1	Characteristics of an Ideal Multi-Tier Tool	2
2	Client/Server Application Design	2
2.1	Rapid Prototyping – From Prototype to Working System	2
3	Multi-Tier Computing: Overview	2
3.1	Definition and Purpose	2
3.2	Key Concepts	2
4	Five-Tier Model (Java Enterprise Application Architecture)	3
5	Three-Tier Architecture	3
5.1	Client Tier	3
5.2	Presentation Tier	3
5.3	Business Tier	4
5.4	Integration Tier	4
5.5	Resource Tier	4
5.6	Tier Interactions (Java EE Example)	4
6	Merits and Demerits of Tier Architectures	5
6.1	1-Tier Architecture	5
6.2	2-Tier Architecture	5
6.3	3-Tier Architecture	6
6.4	N-Tier Architecture	7
7	Benefits and Disadvantages of Multi-Tier Architecture (General)	7
7.1	Benefits	7
7.2	Disadvantages	7
8	Thin Client Computing	7
8.1	Definition	7
8.2	Server Roles and Infrastructure	7
8.3	Thin Client Hardware	8
8.4	Thin Client Software	8
8.5	Thin Client Platform Characteristics	8

8.6 User Scenarios for Thin Client Deployment	8
---	---

Characteristics of an Ideal Multi-Tier Tool

An ideal tool for multi-tier client/server development should include:

- **A visual place-builder for the first tier** – using ActiveX, Java Bean, or Applet technologies.
- **A business-object builder for the second tier** – using ORB, CORBA, Java RMI, or EJB.
- **Frameworks that encapsulate the third tier** – including DBMSs, groupware, TP Monitors, MOM, ERP systems, e-mail, LDAP directories, and workflow.
- **A visual environment for assembling components:** To associate a client event in the first tier with a CORBA or COM+ method invocation on a business object in the second tier. Example: CORBA IDL and Interface Repository.
- **A component repository:** A shared repository of reusable components, places, and server objects. Must provide check-in/check-out facilities and other such features. Example: CORBA object server and interface repository.

Client/Server Application Design

Rapid Prototyping – From Prototype to Working System

Rapid Prototyping is essential. The steps to move from prototype to working system are:

1. Understand the business process.
2. Design the visual place.
3. Define the components or business objects.
4. Work on the detailed object views.
5. Develop dry-run scenarios.
6. Walk through a use-case system scenario.
7. Identify transaction sources.
8. Publish the interfaces for the middle-tier server objects.
9. Develop your code, one business object at a time.
10. Move from prototype to working system.

Multi-Tier Computing: Overview

Definition and Purpose

- Multi-tier architecture divides an application into **multiple layers**, each handling specific tasks.
- **Purpose:**
 - Improve scalability
 - Enhance security
 - Simplify maintenance
 - Support distributed processing

Key Concepts

- Large distributed client/server computing architecture is viewed in terms of **tiers**.

- A **tier** is a logical partition of the separation of concerns in the system.
- Each tier is assigned its **unique responsibility** in the system.
- Each tier is **logically separated** from one another.
- Each tier is **loosely coupled** with the adjacent tier.
- The whole system is represented as a **stack of tiers**.

Five-Tier Model (Java Enterprise Application Architecture)

The Java Enterprise Application Architecture uses a **Five Tier Model** for logical separation of concerns:

Tier	Technologies	Responsibilities
Client Tier	Application clients, applets, apps, and other GUIs	User interaction, UI presentation, devices
Presentation Tier	JSP, Servlets and other UI elements	Single sign-on, session management, content creation, format and delivery
Business Tier	EJBs and other Business Objects	Business logic, transactions, data, services
Integration Tier	JMS, JDBC, Connectors, and Legacy	Resource adapters, legacy, external systems, rules engines, workflow
Resource Tier	Databases, external systems, and legacy resources	Resources, data and external services

Three-Tier Architecture

The classic three-tier architecture consists of:

1. **Presentation Tier** (Client Layer)
2. **Application Tier** (Business Logic)
3. **Data Tier** (Database Server)

Client Tier

- Represents all device or system clients accessing the system or application.
- A client can be: a **Web browser**, a Java or other application, a **Java applet**, a **WAP phone**, a network application, or some future device.
- It could even be a **batch process**.

Presentation Tier

- Encapsulates all **presentation logic** required to service the clients that access the system.
- Responsibilities of the presentation tier:
 - Intercepts client requests
 - Provides **single sign-on**

- Conducts **session management**
- Controls access to business services
- Constructs the responses
- Delivers responses to the client
- **Servlets** and **JSP** reside in this tier.
- Note: Servlets and JSP are not themselves UI elements, but they *produce* UI elements.

Business Tier

- Provides the **business services** required by application clients.
- Contains the **business data** and **business logic**.
- Most business processing for the application is **centralised** into this tier.
- Due to legacy systems, some business processing may occur in the resource tier.
- **Enterprise bean components** (EJBs) are the usual and preferred solution for implementing business objects in this tier.

Integration Tier

- Responsible for communicating with **external resources and systems** such as data stores and legacy applications.
- The business tier is coupled with the integration tier whenever business objects require data or services that reside in the resource tier.
- Components in this tier can use **JDBC**, **J2EE connector technology**, or some proprietary middleware to work with the resource tier.

Resource Tier

- Contains the **business data** and **external resources**, including:
 - Mainframes and legacy systems
 - Business-to-business (B2B) integration systems
 - Services such as credit card authorization

Tier Interactions (Java EE Example)

In a typical Java EE tier interaction:

- **Clients** (Browser, Java Applet, Application) connect to the **Presentation Tier** (Web/HTTP Server hosting JSPs and Servlets).
- The Presentation Tier communicates with the **Business Tier** (Application Server / WebLogic) via **JNDI** or **RMI**.
- The Business Tier hosts **EJBs** that connect to back-end systems:
 - Legacy Systems via **JMS**
 - Databases via **JDBC**
 - Distributed CORBA Objects via **Java IDL**

Merits and Demerits of Tier Architectures

1-Tier Architecture

Advantages	Disadvantages
Easy to implement and optimize performance.	Does not support remote/distributed access for data resources.
No compatibility or context switching issues.	Monolithic manner of the code causes higher maintenance.
Low cost of deployment (development and management).	The cost of the central mainframe is high.

2-Tier Architecture

Advantages	Disadvantages
Applications can be easily developed due to simplicity.	Heterogeneous/rapidly changing business environments are not suitable, since the database server handles business logic, slowing database performance.
Maximum user satisfaction gained with accurate and fast prototyping through robust tools.	Since clients hold most application logic, controlling software versions and redistributing new versions becomes problematic.
More applicable for homogeneous environments due to static business rules.	Security is complicated as users need separate login information for every SQL server.
Database server and business logic are physically close, offering higher performance.	Client tools and SQL middleware in 2-tier environment are proprietary, raising long-term feasibility concerns.
	Scalability: Lacks scalability; supports only a limited number of users; application performance degrades rapidly as simultaneous client requests increase.
	Minimal Logic Sharing: Application logic coupled with client makes it difficult to reuse logic across applications.
	Dispersion of Applications: Any change must reflect across all clients, causing substantial administrative overhead for large user bases.

	Change of Database Structure: Most applications depend on the database structure, creating issues during redesign.
--	--

3-Tier Architecture

Advantages	Disadvantages
Improved Scalability: Distributed deployment of application servers; separate connections from each client not required; TP monitors or ORBs used for load balancing and synchronising servers.	Complexity of Communication: More effort required; communication points are increased (client → middle tier → server) reducing the performance benefit of tools like Visual Basic, PowerBuilder, Delphi.
Enhanced Re-usage: Similar logic can be shared across many clients/applications; object standards like COM/DCOM or CORBA make the business-logic tier language transparent.	Fewer Tools: Additional effort required; 2-tier model can handle certain functions using automated tools and productive reporting tools.
Improved Data Integrity: Data corruption through client applications eliminated; data passed through the middle tier ensures its validity before database updates.	Expert and skilled designers and developers are needed.
Enhanced Security: Multiple layers enhance data security on a service-by-service basis; clients do not interact with the database directly; business logic centralised on server makes data more secure.	More emphasis on architecture and design, requiring domain and design expertise.
Reduced Distribution: Layered architecture allows updating only the application servers, not all distributed clients, when business logic changes.	
Redundant Server Availability: Mission-critical applications can use redundant application and database servers for recovery from network or server failures.	

N-Tier Architecture

Advantages	Disadvantages
Includes all advantages of the 3-tier model (as it is an expansion of the 3-tier methodology). Performance is increased by off-loading from the database tier and client tier, enabling it to suit medium to high volume industries.	Due to the componentisation of the tiers, the complex structure is difficult to implement or maintain.

Benefits and Disadvantages of Multi-Tier Architecture (General)

Benefits

- Scalability
- Improved performance
- Better security
- Easy maintenance
- Reusability of components
- Fault isolation

Disadvantages

- Increased complexity
- Higher development cost
- Network latency
- Deployment challenges
- Requires skilled administration

Thin Client Computing

Definition

- A **thin client** is a lightweight computer purposely built for **remoting into a server** (typically desktop virtualisation resources).
- It depends heavily on another computer (its **server**) to fulfil its computational roles.
- This is different from the traditional **fat client** (desktop PC), which is designed to take on these roles by itself.

Server Roles and Infrastructure

- The specific roles assumed by the server may vary, including:
 - Hosting a shared set of virtualised applications
 - Hosting a shared desktop stack or virtual desktop
 - Data processing and file storage on the client/user's behalf
- Thin clients occur as **components of a broader computing infrastructure**.
- The server-side infrastructure makes use of **cloud computing software** such as:

- Application virtualisation
- Hosted Shared Desktop (HSD)
- Desktop Virtualisation (VDI)
- Benefits of centralisation: **hardware resource optimisation**, **reduced software maintenance**, and **improved security**.

Thin Client Hardware

- Thin client hardware generally consists of a **computer terminal** providing I/O for:
 - Keyboard, mouse, monitor
 - Jacks for sound peripherals
 - Open ports for USB devices (e.g., printer, flash drive, webcam, card reader, smartphone)
- Some thin clients include **legacy serial and/or parallel ports** to support older devices such as receipt printers, scales, time clocks, etc.

Thin Client Software

Thin client software typically consists of:

- A **GUI** (graphical user interface)
- **Cloud access agents** (e.g., RDP, ICA, PCoIP)
- A local **web browser**
- **Terminal emulations** (in some cases)
- A basic set of **local utilities**

Thin Client Platform Characteristics

Platform (Server OS)	Display Encoding	Screen Updates	Client Caching	Client Cache Size	Transport Protocol	Client Display
Microsoft Terminal Services (WinNT, Win2K)	Compressed graphics	Server buffer, pushed adaptively	Glyphs, small bitmaps in memory; large bitmaps on disk	1.5 MB RAM, 10MB disk in Win2K	TCP/IP	Up to 256 colors
Citrix Metaframe (WinNT, Win2K)	Compressed graphics	Server buffer, pushed adaptively	Glyphs, small bitmaps in memory; large bitmaps on disk	3MB RAM, Percent of disk (1% default)	TCP/IP	Up to 24-bit color
LapLink 2000 (WinNT)	DDI graphics, sent to client GDI	Each DDI call sent separately	Glyphs, small bitmaps	1024 objects	TCP/IP	256 colors
AT&T VNC (Linux)	2D run-length encoded pixels	Screen differences sent on client pull	None	N/A	TCP/IP	Up to 32-bit color
Sun Ray (Solaris)	Compressed pixels	Update pushed on each window system command	None	N/A	UDP/IP	Up to 24-bit color

User Scenarios for Thin Client Deployment

The following table compares user types, their tasks, devices, applications, and the proportion of thin-client vs. locally-run applications:

User Type	Tasks, Settings, Devices	Tasks / Applications	App / Job Titles	Thin-Client vs. Local (%)
Mobile task-based user	Handheld or mobile thin-client device	Writing traffic tickets, monitoring car rental returns, taking restaurant orders, reading gas/water	Line-of-business applications	0% local / 100% thin-client
Office task-based user	Windows terminal, NetPC, or network marketers, insurance claim	Traditional office environments: bank tellers, tele-application adjusters, fast food service, automobile service depts., information kiosks	Word processing, line-of-business applications, information	0% local / 100% thin-client
Knowledge workers	Notebook PC and/or desktop PC, handheld devices	Project management, financial reporting, proposal writing, sales, presentations, e-mail, scheduling, contact management	Productivity applications; groupware and communications; Internet, intranet, and extranet information; documents and applications	50% local / 50% thin-client
Power users with creative, analytic, or engineering task needs	Notebook PC and/or desktop PC, and/or high-end workstation; handheld devices	Knowledge worker tasks, plus statistical/scientific analysis, research, periodical or book publishing, graphics, multimedia and Web authoring, software application development	All software used by knowledge worker plus desktop publishing, graphics software, and software development tools	70% local / 30% thin-client

Key insight from user scenario graph: From 100% of applications for task-specific functions (thin-client) to approximately 30% for the most intense power user, the thin-client computing model helps maximise available computing resources.